

TOSS SUBMISSION RESUME

지원 분야

지원자 직무경험 기술서

서버 · 시스템 · 플랫폼 엔지니어링

컴퓨터/소프트웨어공학 전공

Backend / Systems Engineer

2026.07 졸업 예정

OS 커널, 컴파일러, Spring Boot 백엔드, AI 추론 서버까지 직접 구현하며 **예외 상황에서도 무너지지 않는 시스템 구조**를 설계해 온 엔지니어입니다. 문제를 증상 수준에서 봉합하기보다 실행 순서, 상태 전이, API 규약, 배포 전략까지 내려가 재발 가능한 원인을 제거하는 방식으로 접근합니다. 토스처럼 **신뢰도와 속도를 동시에 요구하는 금융 서비스 환경**에서 안정적인 백엔드와 데이터 기반 의사결정으로 기여하고 싶습니다.

핵심 지표

92.52% ECG 진단 모델 Macro AUC**TPS 66** AWS 환경 동시 요청 처리량**0건** API 응답 규약 파싱 오류**4개** 완수한 핵심 기술 프로젝트

기술 스택

SYSTEMS

C / C++ · OS Kernel · Compiler · Virtual Memory · IPC

BACKEND

Java · Spring Boot 3 · PostgreSQL · Redis · Elasticsearch · MinIO

AI / APP

FastAPI · PyTorch · React Native · Vue 3

INFRA

Docker · Docker Compose · AWS EC2 · GitHub Actions

학력 · 어학

학력 소프트웨어공학 학사
2020.09 – 2026.07 졸업 예정**중국어** TSC Level 6 · 비즈니스 회화 가능**영어** OPIc IH · 기술 문서 독해 및 협업 가능**한국어** 원어민

지원 적합성

- **금융 서비스에 맞는 안정성 사고:** API 응답 규약, 예외 처리, 배포 전략을 사람의 주의가 아니라 시스템 구조로 강제하는 방향을 선호합니다.
- **낮은 레벨부터 서비스 레벨까지 연결:** 커널 IPC, 컴파일러 파서, 백엔드 API, AI 추론 서버를 직접 다루며 추상화 아래에서 문제가 발생하는 방식을 경험했습니다.
- **데이터 기반 협업:** 기술 선택과 팀 내 이견을 감정이 아닌 벤치마크, RFC, 문서화로 정리해 합의 속도를 높인 경험이 있습니다.

핵심 역량

장애를 구조적으로 줄이는 설계

락 없는 IPC 레이스 컨디션, API 응답 규약 불일치, 저사양 EC2 OOM처럼 반복 가능한 문제를 실행 순서, 공통 래핑 계층, 배포 전략 변경으로 해결했습니다.

상태와 예외에 강한 구현

Fault-Tolerant 파서와 전역 응답 래핑 경험을 통해 정상 흐름뿐 아니라 비정상 입력과 중간 상태에서도 시스템이 일관성을 유지하도록 설계합니다.

제한된 자원에서의 운영 감각

1GB RAM EC2 환경에서 AI 컨테이너와 백엔드를 분리 운영하고, Pull-only 배포와 컨테이너별 자원 제한으로 안정성을 확보했습니다.

업무 방식

- 문제가 생기면 로그와 재현 조건을 먼저 고정하고, 임시 수정 전에 원인 가설을 좁힙니다.
- 팀 합의가 필요한 사안은 RFC와 벤치마크 수치를 함께 제시해 결정 비용을 줄입니다.
- 구현 후에는 성능 수치, 예외 케이스, 배포 가능성을 함께 확인해 운영 가능한 형태로 마무리합니다.

직무경험 기술서

Projects 01 / 02 · Systems & Compiler

01 MIPS 마이크로커널 운영체제

2024.03 – 2024.07 · 1인 개발

OS Kernel 전체 구현 · 7단계 커널 완주 · Lock-free IPC race condition 해결

MIPS 아키텍처 위에서 부팅, 가상 메모리, 프로세스, 시스템콜, IPC, 파일 시스템까지 OS 핵심 구성요소를 직접 구현하며 커널 레벨 동시성 제어 원리를 체득했습니다.

- 구현**
- 2단계 페이지 테이블과 TLB Fast Refill 핸들러를 어셈블리로 최적화하고, Copy-on-Write 기반 `fork()` 를 구현해 프로세스 생성 시 메모리 오버헤드를 낮췄습니다.
 - 사용자 공간에서 동작하는 FS 서버 프로세스와 공유 페이지 기반 Pipe IPC를 구현해 커널과 파일 시스템을 물리적으로 분리했습니다.
- 문제**
- Pipe IPC 구현 중 스케줄링 타이밍에 따라 열린 파이프를 닫힌 것으로 오인하는 동시성 문제가 발생했습니다. 인터럽트 비활성화 대신 근본 원인인 **매핑 순서 의존성**을 분석했습니다.
- 해결**
- `close()` 와 `dup()` 호출 시 페이지 map/unmap 순서를 재배치해 `pageref(fd) ≤ pageref(pipe)` 불변식이 항상 유지되도록 설계했습니다. 그 결과 락 없이도 안정적인 동시성 제어를 구현했습니다.
- 기여**
- 부팅부터 쉘까지 전 단계를 1인 단독 설계 및 구현했습니다. 특히 코드 실행 순서 재배치만으로 불변식을 강제하는 구조적 해결책을 직접 도출했습니다.

C · MIPS Assembly · GxEmul Simulator · Microkernel FS · Pipe IPC · Virtual Memory

02 SysY 언어 컴파일러 프론트엔드 & VM

2024.10 – 2025.01 · 1인 개발

Lexer / Parser / Symbol Table / P-Code VM 구축 · a-m 전 에러 범주 처리 · Fault-Tolerant 파서

SysY 언어의 Lexer, 재귀 하향 Parser, Symbol Table, P-Code VM 전 과정을 C++17로 구현하고, 구문 오류가 발생해도 파싱을 중단하지 않는 결함 허용 파서를 설계했습니다.

- 구현**
- 모호성을 해결하는 재귀 하향 파서를 설계하고, 스코프 체인을 지원하는 심볼 테이블로 함수 시그니처 불일치와 중복 선언을 컴파일 타임에 진단했습니다.
 - 단일 파일 구조를 Lexer, Parser, SymbolTable, ErrorHandler, Interpreter 5개 모듈로 분리해 에러 복구 로직의 변경 영향 범위를 줄였습니다.
- 문제**
- 세미콜론 누락 에러 복구 시 토큰 포인터가 과도하게 이동해 정상 구문까지 연쇄적으로 오류로 판정하는 문제가 발생했습니다.
- 해결**
- 에러를 파싱 중단 사유가 아닌 **문맥 복귀 신호**로 재정의하고, 직전 유효 토큰 위치인 `mLastNonT` 를 기록해 파서를 중단하지 않고 정상 흐름으로 복구하도록 개선했습니다.
- 학습**
- 견고한 시스템은 정상 입력뿐 아니라 비정상 입력에서도 상태를 잃지 않아야 한다는 점을 체득했습니다. 이후 백엔드 API 예외 처리 구조를 설계할 때 같은 원칙을 적용했습니다.

C++17 · CMake · Lexer / Parser · P-Code VM · Symbol Table · Error Recovery

직무경험 기술서

Projects 03 / 04 · Backend & AI Platform

03 학술 성과 공유 플랫폼

2025.03 – 2025.08 · 백엔드 전담

5인 팀 프로젝트 · 다중 인프라 통합 백엔드 · API 파싱 오류 0건 수렴 · 기한 내 런칭

한국인 1명, 중국인 3명, 교환학생 1명으로 구성된 다국적 팀에서 PostgreSQL, Elasticsearch, Redis, MinIO를 통합한 학술 성과 검색 및 분석 백엔드를 구축했습니다.

- 구현**
- 팀원 간 OS와 DB 버전 파편화로 인한 인프라 기동 실패를 줄이기 위해 Docker Compose 기반 통합 개발 환경을 구축하고 README와 RFC 문서화를 주도했습니다.
 - 검색 성능 요구에 맞춰 PostgreSQL과 Elasticsearch를 함께 운용하고, Redis와 MinIO를 연결해 데이터 조회와 파일 저장 흐름을 분리했습니다.
- 문제**
- 개발자마다 API 응답 구조가 달라 프론트엔드에서 파싱 오류가 반복됐습니다. 문서화와 구두 합의만으로는 런칭 기한이 가까워질수록 휴먼 에러가 재발했습니다.
- 해결**
- Spring의 `ResponseBodyAdvice` 를 구현해 컨트롤러 반환값을 항상 `{code, message, data}` 구조로 자동 래핑했습니다. 규약 준수를 사람에게 맡기지 않고 아키텍처 계층에서 강제해 API 파싱 오류를 0건으로 수렴시켰습니다.
- 협업**
- Vue와 React, Huawei Cloud와 AWS 선택 이견이 있을 때 더미 데이터 10만 건으로 RDBMS LIKE 연산과 Elasticsearch 응답 속도를 ms 단위로 비교해 데이터 기반 합의를 이끌었습니다.

Spring Boot 3 · PostgreSQL · Elasticsearch · Redis · MinIO · Vue 3 · Docker Compose

04 CareLink — 실시간 노인 심전도(ECG) 진단 플랫폼

2025.10 – 2026.03 · 1인 개발 / 졸업논문

3-Tier 아키텍처 · AI 추론 서버 · ECG 진단 AUC 92.52% · TPS 66 AWS 운영

환자와 보호자를 연결하는 모바일 앱과 실시간 ECG AI 진단 기능을 React Native, Spring Boot, FastAPI 3-Tier로 분리해 제한된 AWS 인프라에서도 운영 가능한 구조로 구현했습니다.

- 구현**
- PTB-XL 데이터셋 기반으로 12-lead 진폭 특성 36차원을 설계하고 CNN-CBAM-GRU 모델에 주입해 Macro AUC 92.52%를 달성했습니다.
 - 비즈니스 로직은 Spring Boot, AI 추론은 FastAPI로 분리하고 JWT 인증, 자동 로그아웃, Cache-First 전략으로 앱 응답성을 개선했습니다.
- 문제**
- AI 컨테이너와 백엔드를 동시에 구동하면 1GB RAM EC2에서 OOM이 반복됐습니다. JVM 힙 튜닝만으로는 절대적인 메모리 부족을 해결할 수 없었습니다.
- 해결**
- 로컬과 CI에서 이미지를 빌드해 레지스트리에 push하고, 서버는 Pull-only로만 운영하는 전략으로 전환했습니다. Docker `mem_limit` 으로 컨테이너별 자원을 격리하고 AI 컨테이너는 `profile` 로 분리해 필요할 때만 기동하도록 개선했습니다.
- 기여**
- 프론트, 백엔드, AI 추론 3개 레이어의 설계·구현·배포 전 과정을 1인 단독 수행했습니다. 모델 아키텍처 선정, 피쳐 설계, 손실 함수 조정, 추론 서버 배포까지 데이터 파이프라인 전체를 책임졌습니다.

React Native · Spring Boot 3 · FastAPI · PyTorch · PostgreSQL · AWS EC2 · Docker · GitHub Actions